易學 PyTorch v1.0

入門者必備指南

作者*: jw*

呢本書按類別整理咗 PyTorch 嘅核心函數。每個函數都包含以下內容：

- 代碼例子 - 可運行嘅 Python 代碼
- 運行結果 - 實際輸出結果
- 解釋 - 簡明扼要嘅說明

基於 *PyTorch 2.8.0*

目錄

0. 你好世界
1. 張量創建
2. 基本運算
3. 形狀操作
4. 索引同切片
5. 歸約操作
6. 數學函數
7. 線性代數
8. 神經網絡函數
9. 損失函數
10. 池化同卷積
11. 進階操作
12. 自動微分
13. 設備操作
14. 工具函數
15. 比較操作
16. 張量方法

你好世界

呢一章用一個簡單嘅例子展示完整嘅神經網絡。我哋嚟教一個細嘅神經網絡學 XOR 函數啦。

乜嘢係 **XOR**？

輸入 A	輸入 B	輸出 (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

4 個步驟

1. 數據 - 定義輸入/輸出對
2. 模型 - 創建神經網絡
3. 訓練 - 從數據學習（前向傳播 → 損失 → 反向傳播 → 更新）

4. 推理 - 做預測

完整代碼

```
import torch
import torch.nn as nn

# === 你好世界：超簡單神經網絡 ===
# 目標：學習 XOR 函數 (0^0=0, 0^1=1, 1^0=1, 1^1=0)

# 1. 數據 - 輸入/輸出對
X = torch.tensor([[0., 0.],
                  [0., 1.],
                  [1., 0.],
                  [1., 1.]])

y = torch.tensor([[0.],
                  [1.],
                  [1.],
                  [0.]])

print("=== 數據 ===")
print(f" 輸入 X:\n{X}")
print(f" 答案 y: {y.flatten().tolist()}")

# 2. 模型 - 2 層神經網絡
model = nn.Sequential(
    nn.Linear(2, 4),   # 輸入 2 個 -> 隱藏 4 個
    nn.ReLU(),         # 激活函數
    nn.Linear(4, 1),   # 隱藏 4 個 -> 輸出 1 個
    nn.Sigmoid()       # 輸出 0~1
)

print(f"\n=== 模型 ===")
print(model)

# 3. 訓練
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1.0)

print(f"\n=== 訓練 ===")
for epoch in range(1000):
    pred = model(X)
    loss = criterion(pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch % 200 == 0:
        print(f" 輪次 {epoch:4d}, 損失: {loss.item():.4f}")

# 4. 推理
print(f"\n=== 推理 ===")
with torch.no_grad():
```

```python
    predictions = model(X)
    rounded = (predictions > 0.5).float()

print(" 輸入 -> 預測 -> 四捨五入 -> 答案")
for i in range(4):
    inp = X[i].tolist()
    pred_val = predictions[i].item()
    round_val = int(rounded[i].item())
    target = int(y[i].item())
    status = " 正確" if round_val == target else " 錯誤"
    print(f"{inp} -> {pred_val:.3f} -> {round_val} -> {target} {status}")

accuracy = (rounded == y).float().mean()
print(f"\n準確度: {accuracy.item()*100:.0f}%")
```

運行結果

```
=== 數據 ===
輸入 X:
tensor([[0., 0.],
        [0., 1.],
        [1., 0.],
        [1., 1.]])
答案 y: [0.0, 1.0, 1.0, 0.0]

=== 模型 ===
Sequential(
  (0): Linear(in_features=2, out_features=4, bias=True)
  (1): ReLU()
  (2): Linear(in_features=4, out_features=1, bias=True)
  (3): Sigmoid()
)

=== 訓練 ===
輪次    0, 損失: 0.2455
輪次  200, 損失: 0.1671
輪次  400, 損失: 0.1668
輪次  600, 損失: 0.1668
輪次  800, 損失: 0.1667

=== 推理 ===
輸入 -> 預測 -> 四捨五入 -> 答案
[0.0, 0.0] -> 0.334 -> 0 -> 0 正確
[0.0, 1.0] -> 0.988 -> 1 -> 1 正確
[1.0, 0.0] -> 0.334 -> 0 -> 1 錯誤
[1.0, 1.0] -> 0.334 -> 0 -> 0 正確

準確度: 75%
```

核心概念

- nn.Sequential - 順序堆疊層
- nn.Linear(in, out) - 全連接層

- nn.ReLU() - 激活函數
- nn.MSELoss() - 損失函數（錯幾多？)
- optimizer.zero_grad() - 初始化梯度
- loss.backward() - 計算梯度（反向傳播）
- optimizer.step() - 更新權重
- torch.no_grad() - 推理時禁用梯度

完成！你完成咗第一個神經網絡訓練。

張量創建

torch.tensor(data) 用數據（列表、*numpy* 陣列等）創建張量。自動推斷 *dtype*。

例子:

```
import torch
# Create tensor from Python list
x = torch.tensor([[1, 2], [3, 4]])
print("Tensor from list:")
print(x)
print(f"Shape: {x.shape}, dtype: {x.dtype}")
```

運行結果:

```
Tensor from list:
tensor([[1, 2],
        [3, 4]])
Shape: torch.Size([2, 2]), dtype: torch.int64
```

torch.zeros(*size) 創建填滿零嘅張量。用嚟初始化好有用。

例子:

```
import torch
# Create tensor filled with zeros
x = torch.zeros(2, 3)
print("Zeros tensor (2x3):")
print(x)
```

運行結果:

```
Zeros tensor (2x3):
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

torch.ones(*size) 創建填滿一嘅張量。經常用嚟做遮罩或者初始化。

例子:

```
import torch
# Create tensor filled with ones
x = torch.ones(3, 2)
print("Ones tensor (3x2):")
print(x)
```

運行結果:

```
Ones tensor (3x2):
tensor([[1., 1.],
        [1., 1.],
        [1., 1.]])
```

torch.eye(n) # identity matrix 創建單位矩陣（對角線係 *1*，其他係 *0*）。用於線性代數。

例子:

```python
import torch
# Create identity matrix
x = torch.eye(3)
print("Identity matrix (3x3):")
print(x)
```

運行結果:

```
Identity matrix (3x3):
tensor([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])
```

torch.arange(start, end, step) 創建等間距嘅 *1D* 張量。同 *Python* 嘅 *range()* 類似。

例子:

```python
import torch
# Create range tensor
x = torch.arange(0, 10, 2)
print("Range [0, 10) step 2:")
print(x)
```

運行結果:

```
Range [0, 10) step 2:
tensor([0, 2, 4, 6, 8])
```

torch.linspace(start, end, steps) 創建喺開始同結束之間均勻分佈嘅點嘅張量。

例子:

```python
import torch
# Create linearly spaced tensor
x = torch.linspace(0, 1, 5)
print("Linspace 0 to 1, 5 points:")
print(x)
```

運行結果:

```
Linspace 0 to 1, 5 points:
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

torch.logspace(start, end, steps) 創建對數尺度均勻分佈嘅張量。設置學習率好有用。

例子:

```python
import torch
# Create logarithmically spaced tensor
```

```
x = torch.logspace(0, 2, 3)  # 10^0, 10^1, 10^2
print("Logspace 10^0 to 10^2:")
print(x)
```

運行結果:

```
Logspace 10^0 to 10^2:
tensor([  1.,  10., 100.])
```

torch.rand(*size) # uniform [0,1) 創建 *0* 同 *1* 之間均勻分佈嘅隨機數張量。

例子:

```
import torch
torch.manual_seed(42)
# Create random tensor [0, 1)
x = torch.rand(2, 3)
print("Random uniform [0,1):")
print(x)
```

運行結果:

```
Random uniform [0,1):
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
```

torch.randn(*size) # normal N(0,1) 從標準正態分佈（平均 *=0*，標準差 *=1*）採樣創建張量。

例子:

```
import torch
torch.manual_seed(42)
# Create random tensor from normal distribution
x = torch.randn(2, 3)
print("Random normal N(0,1):")
print(x)
```

運行結果:

```
Random normal N(0,1):
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863]])
```

torch.randint(low, high, size) 創建 *[low, high)* 範圍嘅隨機整數張量。

例子:

```
import torch
torch.manual_seed(42)
# Create random integers
x = torch.randint(0, 10, (2, 3))
print("Random integers [0, 10):")
print(x)
```

運行結果:

```
Random integers [0, 10):
tensor([[2, 7, 6],
```

```
        [4, 6, 5]])
```

torch.empty(*size) 創建未初始化嘅張量。比 *zeros/ones* 快，但會有垃圾值。

例子:

```python
import torch
# Create uninitialized tensor
x = torch.empty(2, 2)
print("Empty tensor (uninitialized):")
print(x)
print("Warning: Contains garbage values!")
```

運行結果:

```
Empty tensor (uninitialized):
tensor([[0., 0.],
        [0., 0.]])
Warning: Contains garbage values!
```

torch.full(size, fill_value) 創建填滿特定值嘅張量。用嚟做常數好有用。

例子:

```python
import torch
# Create tensor filled with specific value
x = torch.full((2, 3), 7.0)
print("Tensor filled with 7.0:")
print(x)
```

運行結果:

```
Tensor filled with 7.0:
tensor([[7., 7., 7.],
        [7., 7., 7.]])
```

torch.zeros_like(x), ones_like(x) 創建同輸入張量相同形狀同 *dtype* 嘅 *0/1* 張量。

例子:

```python
import torch
original = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
zeros = torch.zeros_like(original)
ones = torch.ones_like(original)
print("Original:", original.shape)
print("Zeros like:", zeros)
print("Ones like:", ones)
```

運行結果:

```
Original: torch.Size([2, 2])
Zeros like: tensor([[0., 0.],
        [0., 0.]])
Ones like: tensor([[1., 1.],
        [1., 1.]])
```

基本運算

torch.add(a, b) or a + b 逐元素加法。支持唔同形狀嘅廣播。

例子:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.add(a, b)
print(f"{a} + {b} = {result}")
```

運行結果:

```
tensor([1, 2, 3]) + tensor([4, 5, 6]) = tensor([5, 7, 9])
```

torch.sub(a, b) or a - b 逐元素減法。支持廣播。

例子:

```
import torch
a = torch.tensor([5, 6, 7])
b = torch.tensor([1, 2, 3])
result = torch.sub(a, b)
print(f"{a} - {b} = {result}")
```

運行結果:

```
tensor([5, 6, 7]) - tensor([1, 2, 3]) = tensor([4, 4, 4])
```

torch.mul(a, b) or a * b 逐元素乘法 (*Hadamard* 積)。唔係矩陣乘法。

例子:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.mul(a, b)
print(f"{a} * {b} = {result}")
```

運行結果:

```
tensor([1, 2, 3]) * tensor([4, 5, 6]) = tensor([ 4, 10, 18])
```

torch.div(a, b) or a / b 逐元素除法。用 *float* 張量嚟避免整數除法。

例子:

```
import torch
a = torch.tensor([10.0, 20.0, 30.0])
b = torch.tensor([2.0, 4.0, 5.0])
result = torch.div(a, b)
print(f"{a} / {b} = {result}")
```

運行結果:

```
tensor([10., 20., 30.]) / tensor([2., 4., 5.]) = tensor([5., 5., 6.])
```

torch.matmul(a, b) or a @ b 矩陣乘法。對於 *2D* 張量，執行標準矩陣乘法。

例子:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])
result = torch.matmul(a, b)
print("Matrix A:")
print(a)
print("Matrix B:")
print(b)
print("A @ B:")
print(result)
```

運行結果:

```
Matrix A:
tensor([[1, 2],
        [3, 4]])
Matrix B:
tensor([[5, 6],
        [7, 8]])
A @ B:
tensor([[19, 22],
        [43, 50]])
```

torch.pow(a, exp) or a ** exp 逐元素冪運算。可以用標量或者張量指數。

例子:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
result = torch.pow(x, 2)
print(f"{x} ** 2 = {result}")
```

運行結果:

```
tensor([1., 2., 3.]) ** 2 = tensor([1., 4., 9.])
```

torch.abs(x) # absolute value 返回每個元素嘅絕對值。

例子:

```
import torch
x = torch.tensor([-1, -2, 3, -4])
result = torch.abs(x)
print(f"abs({x}) = {result}")
```

運行結果:

```
abs(tensor([-1, -2,  3, -4])) = tensor([1, 2, 3, 4])
```

torch.neg(x) # negative 返回每個元素嘅負數。同*-x* 一樣。

例子:

```
import torch
x = torch.tensor([1, -2, 3])
result = torch.neg(x)
print(f"neg({x}) = {result}")
```

運行結果:

```
neg(tensor([ 1, -2,  3])) = tensor([-1,  2, -3])
```

`torch.reciprocal(x)  # 1/x` 返回每個元素嘅倒數 (*1/x*)。

例子:

```
import torch
x = torch.tensor([1.0, 2.0, 4.0])
result = torch.reciprocal(x)
print(f"1/{x} = {result}")
```

運行結果:

```
1/tensor([1., 2., 4.]) = tensor([1.0000, 0.5000, 0.2500])
```

`torch.remainder(a, b)  # remainder` 逐元素餘數 (模運算)。

例子:

```
import torch
a = torch.tensor([10, 11, 12])
b = torch.tensor([3, 3, 3])
result = torch.remainder(a, b)
print(f"{a} % {b} = {result}")
```

運行結果:

```
tensor([10, 11, 12]) % tensor([3, 3, 3]) = tensor([1, 2, 0])
```

形狀操作

`x.reshape(*shape)` 返回新形狀嘅張量。元素總數要相同。可能會複製數據。

例子:

```
import torch
x = torch.arange(6)
print(f"Original: {x}")
reshaped = x.reshape(2, 3)
print("Reshaped to (2, 3):")
print(reshaped)
```

運行結果:

```
Original: tensor([0, 1, 2, 3, 4, 5])
Reshaped to (2, 3):
tensor([[0, 1, 2],
        [3, 4, 5]])
```

x.view(*shape) 返回新形狀嘅視圖。需要連續內存。共享數據。

例子:

```
import torch
x = torch.arange(12)
print(f"Original: {x}")
viewed = x.view(3, 4)
print("View as (3, 4):")
print(viewed)
```

運行結果:

```
Original: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
View as (3, 4):
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

x.transpose(dim0, dim1) 交換兩個維度。對於 *2D* 嚟講同矩陣轉置一樣。

例子:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Original (2x3):")
print(x)
transposed = x.transpose(0, 1)
print("Transposed (3x2):")
print(transposed)
```

運行結果:

```
Original (2x3):
tensor([[1, 2, 3],
        [4, 5, 6]])
Transposed (3x2):
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

x.permute(*dims) 重新排列所有維度。比 *transpose* 更靈活。

例子:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Original shape: {x.shape}")
permuted = x.permute(2, 0, 1)
print(f"Permuted shape: {permuted.shape}")
```

運行結果:

```
Original shape: torch.Size([2, 3, 4])
Permuted shape: torch.Size([4, 2, 3])
```

x.squeeze(dim) 移除大小為 *1* 嘅維度。減少張量秩。

例子:

```
import torch
x = torch.zeros(1, 3, 1, 4)
print(f"Original shape: {x.shape}")
squeezed = x.squeeze()
print(f"Squeezed shape: {squeezed.shape}")
```

運行結果:

```
Original shape: torch.Size([1, 3, 1, 4])
Squeezed shape: torch.Size([3, 4])
```

x.unsqueeze(dim) 喺指定位置添加大小為 *1* 嘅維度。

例子:

```
import torch
x = torch.tensor([1, 2, 3])
print(f"Original shape: {x.shape}")
unsqueezed = x.unsqueeze(0)
print(f"Unsqueezed at dim 0: {unsqueezed.shape}")
print(unsqueezed)
```

運行結果:

```
Original shape: torch.Size([3])
Unsqueezed at dim 0: torch.Size([1, 3])
tensor([[1, 2, 3]])
```

x.flatten(start_dim, end_dim) 將張量展平成 *1D*。可選嘅開始/結束維度用於部分展平。

例子:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Original shape: {x.shape}")
flat = x.flatten()
print(f"Flattened: {flat.shape}")
```

運行結果:

```
Original shape: torch.Size([2, 3, 4])
Flattened: torch.Size([24])
```

x.expand(*sizes) 沿住大小為 *1* 嘅維度重複嚟擴展張量。唔複製數據。

例子:

```
import torch
x = torch.tensor([[1], [2], [3]])
print(f"Original: {x.shape}")
expanded = x.expand(3, 4)
print("Expanded to (3, 4):")
print(expanded)
```

運行結果:

```
Original: torch.Size([3, 1])
Expanded to (3, 4):
```

```
tensor([[1, 1, 1, 1],
        [2, 2, 2, 2],
        [3, 3, 3, 3]])
```

x.repeat(*sizes) 沿住每個維度重複張量。創建新內存。

例子:

```
import torch
x = torch.tensor([1, 2])
print(f"Original: {x}")
repeated = x.repeat(3)
print(f"Repeated 3x: {repeated}")
```

運行結果:

```
Original: tensor([1, 2])
Repeated 3x: tensor([1, 2, 1, 2, 1, 2])
```

x.contiguous() 返回內存中連續嘅張量。*transpose* 之後 *view()* 之前需要。

例子:

```
import torch
x = torch.randn(3, 4)
y = x.transpose(0, 1)
print(f"Is contiguous: {y.is_contiguous()}")
z = y.contiguous()
print(f"After contiguous(): {z.is_contiguous()}")
```

運行結果:

```
Is contiguous: False
After contiguous(): True
```

索引同切片

x[i] # index 基本索引，返回索引 *i* 嘅行/元素。

例子:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"x[0] = {x[0]}")
print(f"x[1] = {x[1]}")
```

運行結果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
x[0] = tensor([1, 2, 3])
x[1] = tensor([4, 5, 6])
```

`x[i:j]  # slice` 用 *start:stop:step* 切片。同 *Python* 列表一樣。

例子:

```
import torch
x = torch.arange(10)
print(f"Original: {x}")
print(f"x[2:5] = {x[2:5]}")
print(f"x[::2] = {x[::2]}")
```

運行結果:

```
Original: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
x[2:5] = tensor([2, 3, 4])
x[::2] = tensor([0, 2, 4, 6, 8])
```

`x[..., i]  # ellipsis` 省略號 *(...)* 代表所有剩餘嘅維度。

例子:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"x[..., 0] shape: {x[..., 0].shape}")
print(f"x[0, ...] shape: {x[0, ...].shape}")
```

運行結果:

```
Shape: torch.Size([2, 3, 4])
x[..., 0] shape: torch.Size([2, 3])
x[0, ...] shape: torch.Size([3, 4])
```

`x[:, -1]  # last column` 負索引從尾開始。*-1* 係最後一個元素。

例子:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Last column x[:, -1] = {x[:, -1]}")
```

運行結果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Last column x[:, -1] = tensor([3, 6])
```

`torch.index_select(x, dim, idx)` 用索引張量沿住維度選擇元素。

例子:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
idx = torch.tensor([0, 2])
result = torch.index_select(x, 0, idx)
print("Original:")
print(x)
```

```
print(f"Select rows {idx.tolist()}:")
print(result)
```

運行結果:

```
Original:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
Select rows [0, 2]:
tensor([[1, 2, 3],
        [7, 8, 9]])
```

`torch.masked_select(x, mask)` 返回遮罩為 *True* 嘅元素嘅 *1D* 張量。

例子:

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
mask = x > 2
print(f"Tensor: {x}")
print(f"Mask (>2): {mask}")
print(f"Selected: {torch.masked_select(x, mask)}")
```

運行結果:

```
Tensor: tensor([[1, 2],
        [3, 4]])
Mask (>2): tensor([[False, False],
        [ True,  True]])
Selected: tensor([3, 4])
```

`torch.gather(x, dim, idx)  # gather` 根據索引沿住軸收集值。從分佈中選擇好有用。

例子:

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
idx = torch.tensor([[0, 0], [1, 0]])
result = torch.gather(x, 1, idx)
print("Original:")
print(x)
print(f"Gather with indices {idx.tolist()}:")
print(result)
```

運行結果:

```
Original:
tensor([[1, 2],
        [3, 4]])
Gather with indices [[0, 0], [1, 0]]:
tensor([[1, 1],
        [4, 3]])
```

`torch.scatter(x, dim, idx, src)` 將 *src* 嘅值寫入 *x* 嘅 *idx* 指定嘅位置。*gather* 嘅逆操作。

例子:

```
import torch
x = torch.zeros(3, 5)
idx = torch.tensor([[0, 1, 2, 0, 0], [2, 0, 0, 1, 2]])
src = torch.ones(2, 5)
result = x.scatter(0, idx, src)
print("Scatter result:")
print(result)
```

運行結果:

```
Scatter result:
tensor([[1., 1., 1., 1., 1.],
        [0., 1., 0., 1., 0.],
        [1., 0., 1., 0., 1.]])
```

`torch.where(cond, x, y)  # conditional` 條件為 *True* 嗰度返回 *x* 嘅元素，否則返回 *y* 嘅元素。

例子:

```
import torch
x = torch.tensor([1, 2, 3, 4, 5])
y = torch.tensor([10, 20, 30, 40, 50])
cond = x > 3
result = torch.where(cond, x, y)
print(f"x: {x}")
print(f"y: {y}")
print(f"where(x>3, x, y): {result}")
```

運行結果:

```
x: tensor([1, 2, 3, 4, 5])
y: tensor([10, 20, 30, 40, 50])
where(x>3, x, y): tensor([10, 20, 30,  4,  5])
```

`torch.take(x, indices)  # flat index` 將張量當作 *1D*，返回指定索引嘅元素。

例子:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
idx = torch.tensor([0, 2, 5])
result = torch.take(x, idx)
print(f"Tensor (flattened would be {x.flatten().tolist()})")
print(f"Take indices {idx.tolist()}: {result}")
```

運行結果:

```
Tensor (flattened would be [1, 2, 3, 4, 5, 6])
Take indices [0, 2, 5]: tensor([1, 3, 6])
```

Σ 歸約操作

`x.sum(dim, keepdim)` 求和元素。可選嘅 *dim* 指定軸。*keepdim* 保持維度。

例子:

```python
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Sum all: {x.sum()}")
print(f"Sum dim=0: {x.sum(dim=0)}")
print(f"Sum dim=1: {x.sum(dim=1)}")
```

運行結果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Sum all: 21
Sum dim=0: tensor([5, 7, 9])
Sum dim=1: tensor([ 6, 15])
```

`x.mean(dim, keepdim)` 計算平均值。需要 *float* 張量。

例子:

```python
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print("Tensor:")
print(x)
print(f"Mean all: {x.mean()}")
print(f"Mean dim=1: {x.mean(dim=1)}")
```

運行結果:

```
Tensor:
tensor([[1., 2.],
        [3., 4.]])
Mean all: 2.5
Mean dim=1: tensor([1.5000, 3.5000])
```

`x.std(dim, unbiased)` 計算標準差。*unbiased=True* 用 *N-1* 做分母。

例子:

```python
import torch
x = torch.tensor([1.0, 2.0, 3.0, 4.0, 5.0])
print(f"Data: {x}")
print(f"Std (unbiased): {x.std():.4f}")
print(f"Std (biased): {x.std(unbiased=False):.4f}")
```

運行結果:

```
Data: tensor([1., 2., 3., 4., 5.])
Std (unbiased): 1.5811
Std (biased): 1.4142
```

`x.var(dim, unbiased)` 計算方差（標準差嘅平方）。

例子:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0, 4.0, 5.0])
print(f"Data: {x}")
print(f"Variance: {x.var():.4f}")
```

運行結果:

```
Data: tensor([1., 2., 3., 4., 5.])
Variance: 2.5000
```

`x.max(dim)  # values & indices` 沿住維度返回最大值同索引。

例子:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.max(dim=1)
print(f"Max per row: values={vals}, indices={idxs}")
```

運行結果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Max per row: values=tensor([5, 6]), indices=tensor([1, 2])
```

`x.min(dim)  # values & indices` 沿住維度返回最小值同索引。

例子:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.min(dim=1)
print(f"Min per row: values={vals}, indices={idxs}")
```

運行結果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Min per row: values=tensor([1, 2]), indices=tensor([0, 1])
```

`x.argmax(dim)  # indices only` 返回最大值嘅索引。經常同 *softmax* 輸出一齊用。

例子:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
print(f"Argmax (all): {x.argmax()}")
print(f"Argmax dim=1: {x.argmax(dim=1)}")
```

運行結果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Argmax (all): 5
Argmax dim=1: tensor([1, 2])
```

x.argmin(dim) # indices only 返回最小值嘅索引。

例子:

```python
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
print(f"Argmin dim=1: {x.argmin(dim=1)}")
```

運行結果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Argmin dim=1: tensor([0, 1])
```

x.median(dim) 沿住維度返回中位數同索引。

例子:

```python
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.median(dim=1)
print(f"Median per row: values={vals}, indices={idxs}")
```

運行結果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Median per row: values=tensor([3, 4]), indices=tensor([2, 0])
```

x.mode(dim) 沿住維度返回最頻繁嘅值。

例子:

```python
import torch
x = torch.tensor([[1, 1, 2], [3, 3, 3]])
print("Tensor:")
print(x)
vals, idxs = x.mode(dim=1)
print(f"Mode per row: values={vals}")
```

運行結果:

```
Tensor:
tensor([[1, 1, 2],
        [3, 3, 3]])
Mode per row: values=tensor([1, 3])
```

x.prod(dim) # product 沿住維度計算元素嘅乘積。

例子:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Product all: {x.prod()}")
print(f"Product dim=1: {x.prod(dim=1)}")
```

運行結果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Product all: 720
Product dim=1: tensor([  6, 120])
```

x.cumsum(dim) # cumulative sum 沿住維度計算累積和。每個元素係之前所有元素嘅和。

例子:

```
import torch
x = torch.tensor([1, 2, 3, 4])
print(f"Original: {x}")
print(f"Cumsum: {x.cumsum(dim=0)}")
```

運行結果:

```
Original: tensor([1, 2, 3, 4])
Cumsum: tensor([ 1,  3,  6, 10])
```

x.norm(p, dim) # Lp norm 計算 *Lp* 範數。*L2*（歐幾里得）係默認。*L1* 係曼哈頓距離。

例子:

```
import torch
x = torch.tensor([3.0, 4.0])
print(f"Vector: {x}")
print(f"L2 norm: {x.norm():.4f}")  # sqrt(9+16) = 5
print(f"L1 norm: {x.norm(p=1):.4f}")  # 3+4 = 7
```

運行結果:

```
Vector: tensor([3., 4.])
L2 norm: 5.0000
L1 norm: 7.0000
```

∫ 數學函數

torch.sin(x), cos(x), tan(x) 三角函數。輸入係弧度。

例子:

```
import torch
import math
x = torch.tensor([0, math.pi/2, math.pi])
```

```
print(f"x: {x}")
print(f"sin(x): {torch.sin(x)}")
print(f"cos(x): {torch.cos(x)}")
```

運行結果:

```
x: tensor([0.0000, 1.5708, 3.1416])
sin(x): tensor([ 0.0000e+00,  1.0000e+00, -8.7423e-08])
cos(x): tensor([ 1.0000e+00, -4.3711e-08, -1.0000e+00])
```

torch.asin(x), acos(x), atan(x) 反三角函數。返回弧度。

例子:

```
import torch
x = torch.tensor([0.0, 0.5, 1.0])
print(f"x: {x}")
print(f"asin(x): {torch.asin(x)}")
print(f"acos(x): {torch.acos(x)}")
```

運行結果:

```
x: tensor([0.0000, 0.5000, 1.0000])
asin(x): tensor([0.0000, 0.5236, 1.5708])
acos(x): tensor([1.5708, 1.0472, 0.0000])
```

torch.sinh(x), cosh(x), tanh(x) 雙曲函數。*tanh* 經常用嚟做激活函數。

例子:

```
import torch
x = torch.tensor([-1.0, 0.0, 1.0])
print(f"x: {x}")
print(f"tanh(x): {torch.tanh(x)}")
```

運行結果:

```
x: tensor([-1.,  0.,  1.])
tanh(x): tensor([-0.7616,  0.0000,  0.7616])
```

torch.exp(x), log(x), log10(x) 指數同對數函數。*log* 係自然對數 (底 *e*)。

例子:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"x: {x}")
print(f"exp(x): {torch.exp(x)}")
print(f"log(exp(x)): {torch.log(torch.exp(x))}")
```

運行結果:

```
x: tensor([1., 2., 3.])
exp(x): tensor([ 2.7183,  7.3891, 20.0855])
log(exp(x)): tensor([1.0000, 2.0000, 3.0000])
```

torch.sqrt(x), rsqrt(x) 平方根同倒數平方根（*1/sqrt(x)*）。

例子:

```
import torch
x = torch.tensor([1.0, 4.0, 9.0])
print(f"x: {x}")
print(f"sqrt(x): {torch.sqrt(x)}")
print(f"rsqrt(x): {torch.rsqrt(x)}")  # 1/sqrt(x)
```

運行結果:

```
x: tensor([1., 4., 9.])
sqrt(x): tensor([1., 2., 3.])
rsqrt(x): tensor([1.0000, 0.5000, 0.3333])
```

torch.floor(x), ceil(x), round(x) 取整函數。*floor* 向下取整，*ceil* 向上取整。

例子:

```
import torch
x = torch.tensor([1.2, 2.5, 3.7])
print(f"x: {x}")
print(f"floor(x): {torch.floor(x)}")
print(f"ceil(x): {torch.ceil(x)}")
print(f"round(x): {torch.round(x)}")
```

運行結果:

```
x: tensor([1.2000, 2.5000, 3.7000])
floor(x): tensor([1., 2., 3.])
ceil(x): tensor([2., 3., 4.])
round(x): tensor([1., 2., 4.])
```

torch.clamp(x, min, max) 將值限制喺 *[min, max]* 範圍。超出範圍嘅值設置為邊界值。

例子:

```
import torch
x = torch.tensor([-2, 0, 3, 5, 10])
result = torch.clamp(x, min=0, max=5)
print(f"Original: {x}")
print(f"Clamped [0,5]: {result}")
```

運行結果:

```
Original: tensor([-2,  0,  3,  5, 10])
Clamped [0,5]: tensor([0, 0, 3, 5, 5])
```

torch.sign(x) 根據每個元素嘅符號返回*-1*、*0* 或 *1*。

例子:

```
import torch
x = torch.tensor([-3, 0, 5])
print(f"x: {x}")
print(f"sign(x): {torch.sign(x)}")
```

運行結果:

```
x: tensor([-3,  0,  5])
sign(x): tensor([-1,  0,  1])
```

torch.sigmoid(x) *Sigmoid* 函數：*1/(1+e^-x)*。將值映射到 *(0, 1)*。用於二元分類。

例子:

```
import torch
x = torch.tensor([-2.0, 0.0, 2.0])
print(f"x: {x}")
print(f"sigmoid(x): {torch.sigmoid(x)}")
```

運行結果:

```
x: tensor([-2.,  0.,  2.])
sigmoid(x): tensor([0.1192, 0.5000, 0.8808])
```

線性代數

torch.mm(a, b) # 2D matrix mult *2D* 張量嘅矩陣乘法。批次或更高維用 *matmul*。

例子:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])
result = torch.mm(a, b)
print("A @ B =")
print(result)
```

運行結果:

```
A @ B =
tensor([[19, 22],
        [43, 50]])
```

torch.bmm(a, b) # batch mm 批次矩陣乘法。第一個維度係批次大小。

例子:

```
import torch
a = torch.randn(10, 3, 4)  # batch of 10 matrices
b = torch.randn(10, 4, 5)
result = torch.bmm(a, b)
print(f"Batch shapes: {a.shape} @ {b.shape} = {result.shape}")
```

運行結果:

```
Batch shapes: torch.Size([10, 3, 4]) @ torch.Size([10, 4, 5]) = torch.Size([10, 3, 5])
```

torch.mv(mat, vec) # matrix-vector 矩陣-向量乘法。*vec* 當作列向量。

例子:

```
import torch
mat = torch.tensor([[1, 2], [3, 4]])
vec = torch.tensor([1, 1])
```

```
result = torch.mv(mat, vec)
print(f"Matrix @ vector = {result}")
```

運行結果:

```
Matrix @ vector = tensor([3, 7])
```

`torch.dot(a, b)  # 1D dot product` *1D* 張量嘅點積。逐元素乘積嘅和。

例子:

```
import torch
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
result = torch.dot(a, b)
print(f"{a} . {b} = {result}")
```

運行結果:

```
tensor([1., 2., 3.]) . tensor([4., 5., 6.]) = 32.0
```

`torch.det(x)  # determinant` 計算方陣嘅行列式。

例子:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
det = torch.det(x)
print("Matrix:")
print(x)
print(f"Determinant: {det:.4f}")
```

運行結果:

```
Matrix:
tensor([[1., 2.],
        [3., 4.]])
Determinant: -2.0000
```

`torch.inverse(x)  # matrix inverse` 計算矩陣嘅逆矩陣。*A @ A^-1* = 單位矩陣。

例子:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
inv = torch.inverse(x)
print("Original:")
print(x)
print("Inverse:")
print(inv)
print("X @ X^-1:")
print(torch.mm(x, inv))
```

運行結果:

```
Original:
tensor([[1., 2.],
        [3., 4.]])
```

```
Inverse:
tensor([[-2.0000,  1.0000],
        [ 1.5000, -0.5000]])
X @ X^-1:
tensor([[1., 0.],
        [0., 1.]])
```

`torch.svd(x)  # singular value decomp` 奇異值分解。*X = U @ diag(S) @ V^T*

例子:

```python
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
U, S, V = torch.svd(x)
print(f"Original shape: {x.shape}")
print(f"U: {U.shape}, S: {S.shape}, V: {V.shape}")
print(f"Singular values: {S}")
```

運行結果:

```
Original shape: torch.Size([3, 2])
U: torch.Size([3, 2]), S: torch.Size([2]), V: torch.Size([2, 2])
Singular values: tensor([9.5255, 0.5143])
```

`torch.eig(x)  # eigenvalues` 計算特徵值。一般用 *linalg.eig*，對稱用 *linalg.eigvalsh*。

例子:

```python
import torch
x = torch.tensor([[1.0, 2.0], [2.0, 1.0]])
eigenvalues = torch.linalg.eigvalsh(x)  # For symmetric matrices
print("Matrix:")
print(x)
print(f"Eigenvalues: {eigenvalues}")
```

運行結果:

```
Matrix:
tensor([[1., 2.],
        [2., 1.]])
Eigenvalues: tensor([-1.,  3.])
```

`torch.linalg.norm(x, ord)` 計算矩陣或向量範數。*Frobenius* 係矩陣嘅默認。

例子:

```python
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print("Matrix:")
print(x)
print(f"Frobenius norm: {torch.linalg.norm(x):.4f}")
print(f"L1 norm: {torch.linalg.norm(x, ord=1):.4f}")
```

運行結果:

```
Matrix:
tensor([[1., 2.],
        [3., 4.]])
```

```
Frobenius norm: 5.4772
L1 norm: 6.0000
```

`torch.linalg.solve(A, b)` 解線性方程組 $Ax = b$。比計算逆矩陣更穩定。

例子:

```python
import torch
A = torch.tensor([[3.0, 1.0], [1.0, 2.0]])
b = torch.tensor([9.0, 8.0])
x = torch.linalg.solve(A, b)
print(f"Solving Ax = b")
print(f"x = {x}")
print(f"Verify A@x = {A @ x}")
```

運行結果:

```
Solving Ax = b
x = tensor([2., 3.])
Verify A@x = tensor([9., 8.])
```

`torch.trace(x)  # sum of diagonal` 對角元素嘅和。單位矩陣嘅 *trace* 係維度。

例子:

```python
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Matrix:")
print(x)
print(f"Trace: {torch.trace(x)}")  # 1+5+9 = 15
```

運行結果:

```
Matrix:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
Trace: 15
```

`torch.outer(a, b)  # outer product` 兩個向量嘅外積。結果形狀係 *(len(a), len(b))*。

例子:

```python
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5])
result = torch.outer(a, b)
print(f"a: {a}, b: {b}")
print("Outer product:")
print(result)
```

運行結果:

```
a: tensor([1, 2, 3]), b: tensor([4, 5])
Outer product:
tensor([[ 4,  5],
        [ 8, 10],
        [12, 15]])
```

神經網絡函數

激活函數為神經網絡添加非線性。

F.relu(x) # max(0, x) *ReLU* (整流線性單元)。返回 *max(0, x)*。最常用嘅激活函數。

例子:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"ReLU:  {F.relu(x)}")
```

運行結果:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
ReLU:  tensor([0., 0., 0., 1., 2.])
```

F.leaky_relu(x, neg_slope) 類似 *ReLU* 但允許細嘅負值。防止'*dying ReLU*'問題。

例子:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"Leaky ReLU: {F.leaky_relu(x, 0.1)}")
```

運行結果:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
Leaky ReLU: tensor([-0.2000, -0.1000,  0.0000,  1.0000,  2.0000])
```

F.gelu(x) # Gaussian Error 高斯誤差線性單元。用於 *Transformer* (*BERT, GPT*)。

例子:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"GELU:  {F.gelu(x)}")
```

運行結果:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
GELU:  tensor([-0.0455, -0.1587,  0.0000,  0.8413,  1.9545])
```

F.sigmoid(x) # 1/(1+e^-x) 將值映射到 *(0, 1)*。用於二元分類輸出。

例子:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, 0.0, 2.0])
```

```
print(f"Input: {x}")
print(f"Sigmoid: {F.sigmoid(x)}")
```

運行結果:

```
Input: tensor([-2.,  0.,  2.])
Sigmoid: tensor([0.1192, 0.5000, 0.8808])
```

`F.tanh(x)  # hyperbolic tan` 將值映射到 *(-1, 1)*。以 *0* 為中心，喺隱藏層比 *sigmoid* 好。

例子:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, 0.0, 2.0])
print(f"Input: {x}")
print(f"Tanh: {F.tanh(x)}")
```

運行結果:

```
Input: tensor([-2.,  0.,  2.])
Tanh: tensor([-0.9640,  0.0000,  0.9640])
```

`F.softmax(x, dim)  # probabilities` 將 *logits* 轉換為概率（和為 *1*）。用於多類輸出。

例子:

```
import torch
import torch.nn.functional as F
logits = torch.tensor([2.0, 1.0, 0.1])
probs = F.softmax(logits, dim=0)
print(f"Logits: {logits}")
print(f"Softmax: {probs}")
print(f"Sum: {probs.sum():.4f}")
```

運行結果:

```
Logits: tensor([2.0000, 1.0000, 0.1000])
Softmax: tensor([0.6590, 0.2424, 0.0986])
Sum: 1.0000
```

`F.log_softmax(x, dim)` *softmax* 嘅對數。數值上更穩定。同 *NLLLoss* 一齊用。

例子:

```
import torch
import torch.nn.functional as F
logits = torch.tensor([2.0, 1.0, 0.1])
log_probs = F.log_softmax(logits, dim=0)
print(f"Logits: {logits}")
print(f"Log softmax: {log_probs}")
```

運行結果:

```
Logits: tensor([2.0000, 1.0000, 0.1000])
Log softmax: tensor([-0.4170, -1.4170, -2.3170])
```

正則化技術防止訓練期間過擬合。

F.dropout(x, p, training) 以概率 *p* 將元素隨機設為 *0*。只喺訓練時激活。

例子:

```
import torch
import torch.nn.functional as F
torch.manual_seed(42)
x = torch.ones(10)
dropped = F.dropout(x, p=0.5, training=True)
print(f"Original: {x}")
print(f"Dropout (p=0.5): {dropped}")
```

運行結果:

```
Original: tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
Dropout (p=0.5): tensor([2., 2., 2., 2., 0., 2., 0., 0., 2., 2.])
```

F.batch_norm(x, ...) # normalize 沿住批次維度正則化。穩定訓練。

例子:

```
import torch
import torch.nn.functional as F
x = torch.randn(2, 3, 4, 4)  # batch, channels, H, W
mean = torch.zeros(3)
var = torch.ones(3)
result = F.batch_norm(x, mean, var)
print(f"Input shape: {x.shape}")
print(f"After batch_norm: mean~{result.mean():.4f}, std~{result.std():.4f}")
```

運行結果:

```
Input shape: torch.Size([2, 3, 4, 4])
After batch_norm: mean~0.1266, std~0.9259
```

F.layer_norm(x, shape) 沿住指定維度正則化。用於 *Transformer*。

例子:

```
import torch
import torch.nn.functional as F
x = torch.randn(2, 3, 4)
result = F.layer_norm(x, [3, 4])
print(f"Input shape: {x.shape}")
print(f"After layer_norm: mean~{result.mean():.4f}, std~{result.std():.4f}")
```

運行結果:

```
Input shape: torch.Size([2, 3, 4])
After layer_norm: mean~0.0000, std~1.0215
```

× 損失函數

F.mse_loss(pred, target) 均方誤差。差嘅平方嘅平均值。用於回歸。

例子:

```python
import torch
import torch.nn.functional as F
pred = torch.tensor([1.0, 2.0, 3.0])
target = torch.tensor([1.5, 2.0, 2.5])
loss = F.mse_loss(pred, target)
print(f"Prediction: {pred}")
print(f"Target: {target}")
print(f"MSE Loss: {loss:.4f}")
```

運行結果:

```
Prediction: tensor([1., 2., 3.])
Target: tensor([1.5000, 2.0000, 2.5000])
MSE Loss: 0.1667
```

`F.l1_loss(pred, target)` 平均絕對誤差。比 *MSE* 對異常值更穩健。

例子:

```python
import torch
import torch.nn.functional as F
pred = torch.tensor([1.0, 2.0, 3.0])
target = torch.tensor([1.5, 2.0, 2.5])
loss = F.l1_loss(pred, target)
print(f"Prediction: {pred}")
print(f"Target: {target}")
print(f"L1 Loss: {loss:.4f}")
```

運行結果:

```
Prediction: tensor([1., 2., 3.])
Target: tensor([1.5000, 2.0000, 2.5000])
L1 Loss: 0.3333
```

`F.cross_entropy(logits, labels)` 結合 *log_softmax* 同 *NLLLoss*。多類分類嘅標準。

例子:

```python
import torch
import torch.nn.functional as F
logits = torch.tensor([[2.0, 0.5, 0.1], [0.1, 2.0, 0.5]])
labels = torch.tensor([0, 1])  # Class indices
loss = F.cross_entropy(logits, labels)
print(f"Logits: {logits}")
print(f"Labels: {labels}")
print(f"Cross Entropy Loss: {loss:.4f}")
```

運行結果:

```
Logits: tensor([[2.0000, 0.5000, 0.1000],
        [0.1000, 2.0000, 0.5000]])
Labels: tensor([0, 1])
Cross Entropy Loss: 0.3168
```

`F.nll_loss(log_probs, labels)` 負對數似然。同 *log_softmax* 輸出一齊用。

例子:

```python
import torch
import torch.nn.functional as F
log_probs = F.log_softmax(torch.tensor([[2.0, 0.5], [0.5, 2.0]]), dim=1)
labels = torch.tensor([0, 1])
loss = F.nll_loss(log_probs, labels)
print(f"NLL Loss: {loss:.4f}")
```

運行結果:

```
NLL Loss: 0.2014
```

F.binary_cross_entropy(pred, target) 二元交叉熵。用於 *sigmoid* 輸出嘅二元分類。

例子:

```python
import torch
import torch.nn.functional as F
pred = torch.tensor([0.8, 0.4, 0.9])  # Probabilities
target = torch.tensor([1.0, 0.0, 1.0])  # Binary labels
loss = F.binary_cross_entropy(pred, target)
print(f"Pred: {pred}")
print(f"Target: {target}")
print(f"BCE Loss: {loss:.4f}")
```

運行結果:

```
Pred: tensor([0.8000, 0.4000, 0.9000])
Target: tensor([1., 0., 1.])
BCE Loss: 0.2798
```

F.kl_div(log_pred, target) *Kullback-Leibler* 散度。測量分佈之間嘅差異。

例子:

```python
import torch
import torch.nn.functional as F
log_pred = F.log_softmax(torch.tensor([0.5, 0.3, 0.2]), dim=0)
target = F.softmax(torch.tensor([0.4, 0.4, 0.2]), dim=0)
loss = F.kl_div(log_pred, target, reduction='sum')
print(f"KL Divergence: {loss:.4f}")
```

運行結果:

```
KL Divergence: 0.0035
```

F.cosine_similarity(x1, x2, dim) 測量向量之間嘅角度。*1* = 同方向，*-1* = 反方向。

例子:

```python
import torch
import torch.nn.functional as F
x1 = torch.tensor([[1.0, 0.0, 0.0]])
x2 = torch.tensor([[1.0, 1.0, 0.0]])
sim = F.cosine_similarity(x1, x2)
print(f"x1: {x1}")
print(f"x2: {x2}")
print(f"Cosine similarity: {sim}")
```

運行結果:

```
x1: tensor([[1., 0., 0.]])
x2: tensor([[1., 1., 0.]])
Cosine similarity: tensor([0.7071])
```

F.triplet_margin_loss(...) 學習嵌入，使相似項接近，唔同項遠離。

例子:

```
import torch
import torch.nn.functional as F
anchor = torch.randn(3, 128)
positive = anchor + 0.1 * torch.randn(3, 128)
negative = torch.randn(3, 128)
loss = F.triplet_margin_loss(anchor, positive, negative, margin=1.0)
print(f"Triplet loss: {loss:.4f}")
```

運行結果:

```
Triplet loss: 0.0000
```

池化同卷積

F.max_pool2d(x, kernel_size) 取每個窗口嘅最大值嚟下採樣。減少空間維度。

例子:

```
import torch
import torch.nn.functional as F
x = torch.arange(16).float().view(1, 1, 4, 4)
print("Input (4x4):")
print(x.squeeze())
result = F.max_pool2d(x, kernel_size=2)
print("MaxPool2d (kernel=2):")
print(result.squeeze())
```

運行結果:

```
Input (4x4):
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.],
        [12., 13., 14., 15.]])
MaxPool2d (kernel=2):
tensor([[ 5.,  7.],
        [13., 15.]])
```

F.avg_pool2d(x, kernel_size) 對每個窗口取平均嚟下採樣。比 *max pooling* 更平滑。

例子:

```
import torch
import torch.nn.functional as F
x = torch.arange(16).float().view(1, 1, 4, 4)
print("Input (4x4):")
```

```
print(x.squeeze())
result = F.avg_pool2d(x, kernel_size=2)
print("AvgPool2d (kernel=2):")
print(result.squeeze())
```

運行結果:

```
Input (4x4):
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.],
        [12., 13., 14., 15.]])
AvgPool2d (kernel=2):
tensor([[ 2.5000,  4.5000],
        [10.5000, 12.5000]])
```

F.adaptive_max_pool2d(x, output_size) 無論輸入大小，都池化到精確嘅輸出大小。

例子:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 7, 7)
result = F.adaptive_max_pool2d(x, output_size=(2, 2))
print(f"Input: {x.shape} -> Output: {result.shape}")
```

運行結果:

```
Input: torch.Size([1, 1, 7, 7]) -> Output: torch.Size([1, 1, 2, 2])
```

F.conv2d(x, weight, bias) *2D* 卷積。*CNN* 嘅核心操作。提取空間特徵。

例子:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 5, 5)  # batch, in_ch, H, W
weight = torch.randn(1, 1, 3, 3)  # out_ch, in_ch, kH, kW
result = F.conv2d(x, weight)
print(f"Input: {x.shape}")
print(f"Weight: {weight.shape}")
print(f"Output: {result.shape}")
```

運行結果:

```
Input: torch.Size([1, 1, 5, 5])
Weight: torch.Size([1, 1, 3, 3])
Output: torch.Size([1, 1, 3, 3])
```

F.conv_transpose2d(x, weight) 轉置卷積（反卷積）。上採樣空間維度。

例子:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 3, 3)
weight = torch.randn(1, 1, 3, 3)
```

```
result = F.conv_transpose2d(x, weight)
print(f"Input: {x.shape}")
print(f"Output: {result.shape}")
```

運行結果:

```
Input: torch.Size([1, 1, 3, 3])
Output: torch.Size([1, 1, 5, 5])
```

`F.interpolate(x, size, mode)` 用插值調整張量大小。模式：*nearest*、*bilinear*、*bicubic*。

例子:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 4, 4)
result = F.interpolate(x, size=(8, 8), mode='bilinear', align_corners=False)
print(f"Input: {x.shape} -> Output: {result.shape}")
```

運行結果:

```
Input: torch.Size([1, 1, 4, 4]) -> Output: torch.Size([1, 1, 8, 8])
```

`F.pad(x, pad, mode)` 喺張量周圍添加填充。用於卷積。模式：*constant*、*reflect*、*replicate*。

例子:

```
import torch
import torch.nn.functional as F
x = torch.arange(4).float().view(1, 1, 2, 2)
print("Input:")
print(x.squeeze())
padded = F.pad(x, (1, 1, 1, 1), mode='constant', value=0)
print("Padded (1 on each side):")
print(padded.squeeze())
```

運行結果:

```
Input:
tensor([[0., 1.],
        [2., 3.]])
Padded (1 on each side):
tensor([[0., 0., 0., 0.],
        [0., 0., 1., 0.],
        [0., 2., 3., 0.],
        [0., 0., 0., 0.]])
```

∞ 進階操作

`torch.einsum('ij,jk->ik', a, b)` 愛因斯坦求和。好多張量操作嘅靈活記法。

例子:

```
import torch
a = torch.randn(2, 3)
b = torch.randn(3, 4)
```

```
result = torch.einsum('ij,jk->ik', a, b)
print(f"einsum('ij,jk->ik'): {a.shape} x {b.shape} = {result.shape}")
# Verify it's matrix multiplication
print(f"Same as matmul: {torch.allclose(result, a @ b)}")
```

運行結果:

```
einsum('ij,jk->ik'): torch.Size([2, 3]) x torch.Size([3, 4]) = torch.Size([2, 4])
Same as matmul: True
```

`torch.topk(x, k, dim)  # top k values` 返回 k 個最大嘅值同索引。比完全排序快。

例子:

```
import torch
x = torch.tensor([1, 5, 3, 9, 2, 7])
vals, idxs = torch.topk(x, k=3)
print(f"Input: {x}")
print(f"Top 3 values: {vals}")
print(f"Top 3 indices: {idxs}")
```

運行結果:

```
Input: tensor([1, 5, 3, 9, 2, 7])
Top 3 values: tensor([9, 7, 5])
Top 3 indices: tensor([3, 5, 1])
```

`torch.sort(x, dim)  # sorted values` 沿住維度對張量排序。返回值同原始索引。

例子:

```
import torch
x = torch.tensor([3, 1, 4, 1, 5, 9])
vals, idxs = torch.sort(x)
print(f"Original: {x}")
print(f"Sorted: {vals}")
print(f"Indices: {idxs}")
```

運行結果:

```
Original: tensor([3, 1, 4, 1, 5, 9])
Sorted: tensor([1, 1, 3, 4, 5, 9])
Indices: tensor([1, 3, 0, 2, 4, 5])
```

`torch.argsort(x, dim)  # sort indices` 返回對張量排序嘅索引。

例子:

```
import torch
x = torch.tensor([3, 1, 4, 1, 5])
idxs = torch.argsort(x)
print(f"Original: {x}")
print(f"Argsort: {idxs}")
print(f"Sorted via indices: {x[idxs]}")
```

運行結果:

```
Original: tensor([3, 1, 4, 1, 5])
Argsort: tensor([1, 3, 0, 2, 4])
Sorted via indices: tensor([1, 1, 3, 4, 5])
```

`torch.unique(x)  # unique values` 返回唯一元素。可選 *return_counts* 獲取頻率。

例子:

```
import torch
x = torch.tensor([1, 2, 2, 3, 1, 3, 3, 4])
unique = torch.unique(x)
print(f"Original: {x}")
print(f"Unique: {unique}")
```

運行結果:

```
Original: tensor([1, 2, 2, 3, 1, 3, 3, 4])
Unique: tensor([1, 2, 3, 4])
```

`torch.cat([t1, t2], dim)  # concat` 沿住現有維度連接張量。除咗 *dim* 之外形狀要相同。

例子:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6]])
result = torch.cat([a, b], dim=0)
print("Concatenated along dim 0:")
print(result)
```

運行結果:

```
Concatenated along dim 0:
tensor([[1, 2],
        [3, 4],
        [5, 6]])
```

`torch.stack([t1, t2], dim)  # new dim` 沿住新維度堆疊張量。所有張量形狀要相同。

例子:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.stack([a, b], dim=0)
print("Stacked (creates new dim):")
print(result)
print(f"Shape: {result.shape}")
```

運行結果:

```
Stacked (creates new dim):
tensor([[1, 2, 3],
        [4, 5, 6]])
Shape: torch.Size([2, 3])
```

torch.split(x, size, dim) # split 將張量分成指定大小嘅塊。最後一塊可能細啲。

例子:

```
import torch
x = torch.arange(10)
splits = torch.split(x, 3)
print(f"Original: {x}")
print("Split into chunks of 3:")
for i, s in enumerate(splits):
    print(f"  {i}: {s}")
```

運行結果:

```
Original: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
Split into chunks of 3:
  0: tensor([0, 1, 2])
  1: tensor([3, 4, 5])
  2: tensor([6, 7, 8])
  3: tensor([9])
```

torch.chunk(x, chunks, dim) # chunks 將張量分成指定數量嘅塊。

例子:

```
import torch
x = torch.arange(12)
chunks = torch.chunk(x, 4)
print(f"Original: {x}")
print("Split into 4 chunks:")
for i, c in enumerate(chunks):
    print(f"  {i}: {c}")
```

運行結果:

```
Original: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
Split into 4 chunks:
  0: tensor([0, 1, 2])
  1: tensor([3, 4, 5])
  2: tensor([6, 7, 8])
  3: tensor([ 9, 10, 11])
```

torch.broadcast_to(x, shape) 明確將張量廣播到新形狀。唔複製數據。

例子:

```
import torch
x = torch.tensor([1, 2, 3])
result = torch.broadcast_to(x, (3, 3))
print(f"Original: {x}")
print("Broadcast to (3, 3):")
print(result)
```

運行結果:

```
Original: tensor([1, 2, 3])
Broadcast to (3, 3):
tensor([[1, 2, 3],
```

```
        [1, 2, 3],
        [1, 2, 3]])
```

torch.flatten(x, start, end) 展平張量。可選嘅開始/結束維度用於部分展平。

例子:

```
import torch
x = torch.randn(2, 3, 4)
flat = torch.flatten(x)
partial = torch.flatten(x, start_dim=1)
print(f"Original: {x.shape}")
print(f"Fully flat: {flat.shape}")
print(f"Flatten from dim 1: {partial.shape}")
```

運行結果:

```
Original: torch.Size([2, 3, 4])
Fully flat: torch.Size([24])
Flatten from dim 1: torch.Size([2, 12])
```

∂ 自動微分

x.requires_grad_(True) 啟用張量嘅梯度追蹤。反向傳播需要。

例子:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"Before: requires_grad = {x.requires_grad}")
x.requires_grad_(True)
print(f"After: requires_grad = {x.requires_grad}")
```

運行結果:

```
Before: requires_grad = False
After: requires_grad = True
```

y.backward() 通過反向傳播計算梯度。存儲喺*.grad* 屬性。

例子:

```
import torch
x = torch.tensor([2.0, 3.0], requires_grad=True)
y = (x ** 2).sum()  # y = x1^2 + x2^2
y.backward()
print(f"x = {x}")
print(f"y = x^2.sum() = {y}")
print(f"dy/dx = {x.grad}")  # 2*x
```

運行結果:

```
x = tensor([2., 3.], requires_grad=True)
y = x^2.sum() = 13.0
dy/dx = tensor([4., 6.])
```

x.grad # gradient *backward()* 後存儲累積嘅梯度。用 *zero_grad()* 重置。

例子:

```
import torch
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
y.backward()
print(f"x = {x}")
print(f"y = x^2 = {y}")
print(f"dy/dx = {x.grad}")
```

運行結果:

```
x = 3.0
y = x^2 = 9.0
dy/dx = 6.0
```

x.detach() 返回從計算圖分離嘅張量。停止梯度流動。

例子:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = x * 2
z = y.detach()
print(f"y requires_grad: {y.requires_grad}")
print(f"z requires_grad: {z.requires_grad}")
```

運行結果:

```
y requires_grad: True
z requires_grad: False
```

x.clone() 創建相同數據嘅副本。修改唔影響原始。

例子:

```
import torch
x = torch.tensor([1.0, 2.0])
y = x.clone()
y[0] = 99
print(f"Original x: {x}")
print(f"Cloned y: {y}")
```

運行結果:

```
Original x: tensor([1., 2.])
Cloned y: tensor([99.,  2.])
```

with torch.no_grad(): 禁用梯度計算嘅上下文管理器。用於推理。

例子:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
with torch.no_grad():
    y = x * 2
    print(f"Inside no_grad: y.requires_grad = {y.requires_grad}")
```

```
z = x * 2
print(f"Outside: z.requires_grad = {z.requires_grad}")
```

運行結果:

```
Inside no_grad: y.requires_grad = False
Outside: z.requires_grad = True
```

`torch.autograd.grad(y, x)` 計算梯度而唔修改*.grad*。對二階導數有用。

例子:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = (x ** 3).sum()  # y = x1^3 + x2^3
grad = torch.autograd.grad(y, x)
print(f"x = {x}")
print(f"dy/dx = {grad[0]}")  # 3*x^2
```

運行結果:

```
x = tensor([1., 2.], requires_grad=True)
dy/dx = tensor([ 3., 12.])
```

`optimizer.zero_grad()` 將梯度重置為 *0*。每次 *backward pass* 之前調用。

例子:

```
import torch
import torch.optim as optim
x = torch.tensor([1.0], requires_grad=True)
optimizer = optim.SGD([x], lr=0.1)
y = x ** 2
y.backward()
print(f"Grad before zero_grad: {x.grad}")
optimizer.zero_grad()
print(f"Grad after zero_grad: {x.grad}")
```

運行結果:

```
Grad before zero_grad: tensor([2.])
Grad after zero_grad: None
```

`optimizer.step()` 用計算嘅梯度更新參數。$x = x - lr$ grad。*

例子:

```
import torch
import torch.optim as optim
x = torch.tensor([5.0], requires_grad=True)
optimizer = optim.SGD([x], lr=0.1)
print(f"Before: x = {x.item():.4f}")
y = x ** 2
y.backward()
optimizer.step()
print(f"After step: x = {x.item():.4f}")
```

運行結果:

```
Before: x = 5.0000
After step: x = 4.0000
```

`torch.nn.utils.clip_grad_norm_(params, max)` 裁剪梯度範數以防止梯度爆炸。*RNN* 必須用。

例子:

```
import torch
import torch.nn as nn
model = nn.Linear(10, 5)
x = torch.randn(1, 10)
loss = model(x).sum()
loss.backward()
norm_before = sum(p.grad.norm()**2 for p in model.parameters())**0.5
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
norm_after = sum(p.grad.norm()**2 for p in model.parameters())**0.5
print(f"Grad norm before: {norm_before:.4f}")
print(f"Grad norm after clip: {norm_after:.4f}")
```

運行結果:

```
Grad norm before: 6.8840
Grad norm after clip: 1.0000
```

設備操作

`torch.cuda.is_available()` 檢查 *CUDA GPU* 係咪可用。用於設備選擇邏輯。

例子:

```
import torch
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"MPS available: {torch.backends.mps.is_available()}")
```

運行結果:

```
CUDA available: False
MPS available: True
```

`torch.device('cuda'/'cpu'/'mps')` 創建設備對象。同*.to(device)* 一齊用於設備放置。

例子:

```
import torch
cpu = torch.device('cpu')
print(f"CPU device: {cpu}")
if torch.cuda.is_available():
    gpu = torch.device('cuda')
    print(f"GPU device: {gpu}")
elif torch.backends.mps.is_available():
    mps = torch.device('mps')
    print(f"MPS device: {mps}")
```

運行結果:

```
CPU device: cpu
MPS device: mps
```

`x.to(device)  # move to device` 將張量移到指定設備。*GPU* 訓練必須用。

例子:

```
import torch
x = torch.tensor([1.0, 2.0])
device = torch.device('cpu')
x_dev = x.to(device)
print(f"Tensor on: {x_dev.device}")
```

運行結果:

```
Tensor on: cpu
```

`x.to(dtype)  # change dtype` 將張量轉換為唔同數據類型。

例子:

```
import torch
x = torch.tensor([1, 2, 3])
print(f"Original dtype: {x.dtype}")
x_float = x.to(torch.float32)
print(f"After to(float32): {x_float.dtype}")
```

運行結果:

```
Original dtype: torch.int64
After to(float32): torch.float32
```

`x.cuda(), x.cpu()  # shortcuts` 喺 *CPU* 同 *CUDA* 之間移動嘅快捷方法。

例子:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Original device: {x.device}")
x_cpu = x.cpu()
print(f"After cpu(): {x_cpu.device}")
```

運行結果:

```
Original device: cpu
After cpu(): cpu
```

`x.device  # check current device` 返回張量存儲嘅設備。

例子:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Device: {x.device}")
print(f"Device type: {x.device.type}")
```

運行結果:

```
Device: cpu
Device type: cpu
```

`x.is_cuda  # boolean check` 如果張量喺 *CUDA* 設備上就返回 *True*。

例子:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"is_cuda: {x.is_cuda}")
```

運行結果:

```
is_cuda: False
```

`torch.cuda.empty_cache()` 釋放未使用嘅緩存 *GPU* 內存。對 *OOM* 錯誤有幫助。

例子:

```
import torch
# Free unused cached memory (only affects CUDA)
if torch.cuda.is_available():
    torch.cuda.empty_cache()
    print("CUDA cache cleared")
else:
    print("CUDA not available - cache clearing skipped")
```

運行結果:

```
CUDA not available - cache clearing skipped
```

`torch.cuda.device_count()` 返回可用 *CUDA* 設備嘅數量。

例子:

```
import torch
if torch.cuda.is_available():
    count = torch.cuda.device_count()
    print(f"Number of GPUs: {count}")
else:
    print("CUDA not available")
```

運行結果:

```
CUDA not available
```

※ 工具函數

`x.dtype, x.shape, x.size()` 基本張量屬性。*shape* 同 *size()* 係一樣嘅。

例子:

```
import torch
x = torch.randn(2, 3, 4)
print(f"dtype: {x.dtype}")
print(f"shape: {x.shape}")
print(f"size(): {x.size()}")
```

運行結果:

```
dtype: torch.float32
shape: torch.Size([2, 3, 4])
size(): torch.Size([2, 3, 4])
```

`x.numel()  # number of elements` 返回元素總數。所有維度嘅乘積。

例子:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"Number of elements: {x.numel()}")
```

運行結果:

```
Shape: torch.Size([2, 3, 4])
Number of elements: 24
```

`x.dim()  # number of dimensions` 返回張量嘅維度數（秩）。

例子:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"Dimensions: {x.dim()}")
```

運行結果:

```
Shape: torch.Size([2, 3, 4])
Dimensions: 3
```

`x.ndimension()  # same as dim()` *dim()* 嘅別名。返回維度數。

例子:

```
import torch
x = torch.randn(2, 3, 4)
print(f"ndimension(): {x.ndimension()}")
print(f"Same as dim(): {x.dim()}")
```

運行結果:

```
ndimension(): 3
Same as dim(): 3
```

`x.is_contiguous()` 檢查張量喺內存中係咪連續。*view()* 需要。

例子:

```
import torch
x = torch.randn(3, 4)
y = x.transpose(0, 1)
print(f"Original is_contiguous: {x.is_contiguous()}")
print(f"Transposed is_contiguous: {y.is_contiguous()}")
```

運行結果:

```
Original is_contiguous: True
Transposed is_contiguous: False
```

x.float(), x.int(), x.long() *dtype* 轉換嘅快捷方法。

例子:

```
import torch
x = torch.tensor([1, 2, 3])
print(f"Original: {x.dtype}")
print(f"float(): {x.float().dtype}")
print(f"long(): {x.long().dtype}")
```

運行結果:

```
Original: torch.int64
float(): torch.float32
long(): torch.int64
```

x.half(), x.double() # fp16, fp64 半精度（*16* 位）用於快速訓練，*double* 用於高精度。

例子:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Original: {x.dtype}")
print(f"half() (fp16): {x.half().dtype}")
print(f"double() (fp64): {x.double().dtype}")
```

運行結果:

```
Original: torch.float32
half() (fp16): torch.float16
double() (fp64): torch.float64
```

torch.from_numpy(arr) 將 *NumPy* 陣列轉換為張量。同原始陣列共享內存。

例子:

```
import torch
import numpy as np
arr = np.array([1, 2, 3])
x = torch.from_numpy(arr)
print(f"NumPy: {arr}, dtype={arr.dtype}")
print(f"Tensor: {x}, dtype={x.dtype}")
```

運行結果:

```
NumPy: [1 2 3], dtype=int64
Tensor: tensor([1, 2, 3]), dtype=torch.int64
```

x.numpy() # CPU only 將張量轉換為 *NumPy* 陣列。必須喺 *CPU* 上。

例子:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
arr = x.numpy()
```

```
print(f"Tensor: {x}")
print(f"NumPy: {arr}, dtype={arr.dtype}")
```

運行結果:

```
Tensor: tensor([1., 2., 3.])
NumPy: [1. 2. 3.], dtype=float32
```

`torch.save(obj, path)` 將張量或模型保存到文件。用 *pickle* 序列化。

例子:

```
import torch
import os
x = torch.tensor([1, 2, 3])
torch.save(x, '/tmp/tensor.pt')
print(f"Saved tensor to /tmp/tensor.pt")
print(f"File size: {os.path.getsize('/tmp/tensor.pt')} bytes")
```

運行結果:

```
Saved tensor to /tmp/tensor.pt
File size: 1570 bytes
```

`torch.load(path)` 從文件加載保存概張量或模型。

例子:

```
import torch
torch.save(torch.tensor([1, 2, 3]), '/tmp/tensor.pt')
loaded = torch.load('/tmp/tensor.pt', weights_only=True)
print(f"Loaded: {loaded}")
```

運行結果:

```
Loaded: tensor([1, 2, 3])
```

`torch.manual_seed(seed)` 設置隨機種子以實現可重現性。

例子:

```
import torch
torch.manual_seed(42)
a = torch.rand(3)
torch.manual_seed(42)
b = torch.rand(3)
print(f"First: {a}")
print(f"Second (same seed): {b}")
print(f"Equal: {torch.equal(a, b)}")
```

運行結果:

```
First: tensor([0.8823, 0.9150, 0.3829])
Second (same seed): tensor([0.8823, 0.9150, 0.3829])
Equal: True
```

≈ 比較操作

torch.eq(a, b) or a == b 逐元素相等比較。返回布爾張量。

例子:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([1, 0, 3])
print(f"a: {a}")
print(f"b: {b}")
print(f"a == b: {a == b}")
```

運行結果:

```
a: tensor([1, 2, 3])
b: tensor([1, 0, 3])
a == b: tensor([ True, False,  True])
```

torch.ne(a, b) or a != b 逐元素不等比較。

例子:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([1, 0, 3])
print(f"a != b: {a != b}")
```

運行結果:

```
a != b: tensor([False,  True, False])
```

torch.gt(a, b) or a > b 逐元素大於比較。

例子:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a: {a}, b: {b}")
print(f"a > b: {a > b}")
```

運行結果:

```
a: tensor([1, 2, 3]), b: tensor([2, 2, 2])
a > b: tensor([False, False,  True])
```

torch.lt(a, b) or a < b 逐元素小於比較。

例子:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a < b: {a < b}")
```

運行結果:

```
a < b: tensor([ True, False, False])
```

torch.ge(a, b) or a >= b 逐元素大於等於比較。

例子:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a >= b: {a >= b}")
```

運行結果:

```
a >= b: tensor([False,  True,  True])
```

torch.le(a, b) or a <= b 逐元素小於等於比較。

例子:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a <= b: {a <= b}")
```

運行結果:

```
a <= b: tensor([ True,  True, False])
```

torch.allclose(a, b, rtol, atol) 檢查所有元素係咪喺容差範圍內接近。用於 *float* 比較。

例子:

```
import torch
a = torch.tensor([1.0, 2.0])
b = torch.tensor([1.0001, 2.0001])
print(f"a: {a}")
print(f"b: {b}")
print(f"allclose (default tol): {torch.allclose(a, b)}")
print(f"allclose (rtol=1e-3): {torch.allclose(a, b, rtol=1e-3)}")
```

運行結果:

```
a: tensor([1., 2.])
b: tensor([1.0001, 2.0001])
allclose (default tol): False
allclose (rtol=1e-3): True
```

torch.isnan(x) 元素係 *NaN*（非數字）嗰度返回 *True*。

例子:

```
import torch
x = torch.tensor([1.0, float('nan'), 3.0])
print(f"x: {x}")
print(f"isnan: {torch.isnan(x)}")
```

運行結果:

```
x: tensor([1., nan, 3.])
isnan: tensor([False,  True, False])
```

torch.isinf(x) 元素係無窮大嗰度返回 *True*。

例子:

```
import torch
x = torch.tensor([1.0, float('inf'), float('-inf')])
print(f"x: {x}")
print(f"isinf: {torch.isinf(x)}")
```

運行結果:

```
x: tensor([1., inf, -inf])
isinf: tensor([False,  True,  True])
```

torch.isfinite(x) 元素係有限（唔係 *inf*，唔係 *nan*）嗰度返回 *True*。

例子:

```
import torch
x = torch.tensor([1.0, float('inf'), float('nan')])
print(f"x: {x}")
print(f"isfinite: {torch.isfinite(x)}")
```

運行結果:

```
x: tensor([1., inf, nan])
isfinite: tensor([ True, False, False])
```

● 張量方法

x.T # transpose (2D) *2D* 轉置嘅簡寫。更高維用 *permute*。

例子:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Original:")
print(x)
print("x.T:")
print(x.T)
```

運行結果:

```
Original:
tensor([[1, 2, 3],
        [4, 5, 6]])
x.T:
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

x.H # conjugate transpose 複數張量嘅共軛轉置。轉置 + 共軛。

例子:

```
import torch
x = torch.tensor([[1+2j, 3+4j], [5+6j, 7+8j]])
print("Original:")
```

```
print(x)
print("x.H (conjugate transpose):")
print(x.H)
```

運行結果:

```
Original:
tensor([[1.+2.j, 3.+4.j],
        [5.+6.j, 7.+8.j]])
x.H (conjugate transpose):
tensor([[1.-2.j, 5.-6.j],
        [3.-4.j, 7.-8.j]])
```

`x.real, x.imag  # complex parts` 訪問複數張量嘅實部同虛部。

例子:

```
import torch
x = torch.tensor([1+2j, 3+4j])
print(f"Complex: {x}")
print(f"Real: {x.real}")
print(f"Imag: {x.imag}")
```

運行結果:

```
Complex: tensor([1.+2.j, 3.+4.j])
Real: tensor([1., 3.])
Imag: tensor([2., 4.])
```

`x.abs(), x.neg()  # absolute, negate` 絕對值同取負嘅方法。

例子:

```
import torch
x = torch.tensor([-3, -1, 0, 2, 4])
print(f"x: {x}")
print(f"abs: {x.abs()}")
print(f"neg: {x.neg()}")
```

運行結果:

```
x: tensor([-3, -1,  0,  2,  4])
abs: tensor([3, 1, 0, 2, 4])
neg: tensor([ 3,  1,  0, -2, -4])
```

`x.reciprocal(), x.pow(n)` 作為方法嘅倒數 (*1/x*) 同冪運算。

例子:

```
import torch
x = torch.tensor([1.0, 2.0, 4.0])
print(f"x: {x}")
print(f"reciprocal: {x.reciprocal()}")
print(f"pow(2): {x.pow(2)}")
```

運行結果:

```
x: tensor([1., 2., 4.])
reciprocal: tensor([1.0000, 0.5000, 0.2500])
pow(2): tensor([ 1.,  4., 16.])
```

`x.sqrt()`, `x.exp()`, `x.log()` 作為張量方法嘅常見數學運算。

例子:

```
import torch
x = torch.tensor([1.0, 4.0, 9.0])
print(f"x: {x}")
print(f"sqrt: {x.sqrt()}")
print(f"exp: {torch.tensor([0.0, 1.0]).exp()}")
print(f"log: {torch.tensor([1.0, 2.718]).log()}")
```

運行結果:

```
x: tensor([1., 4., 9.])
sqrt: tensor([1., 2., 3.])
exp: tensor([1.0000, 2.7183])
log: tensor([0.0000, 0.9999])
```

`x.item()  # get scalar value` 從單元素張量提取標量值為 *Python* 數字。

例子:

```
import torch
x = torch.tensor(3.14159)
val = x.item()
print(f"Tensor: {x}")
print(f"Python float: {val}")
print(f"Type: {type(val)}")
```

運行結果:

```
Tensor: 3.141590118408203
Python float: 3.141590118408203
Type: <class 'float'>
```

`x.tolist()  # to Python list` 將張量轉換為嵌套 *Python* 列表。

例子:

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
lst = x.tolist()
print(f"Tensor: {x}")
print(f"List: {lst}")
print(f"Type: {type(lst)}")
```

運行結果:

```
Tensor: tensor([[1, 2],
        [3, 4]])
List: [[1, 2], [3, 4]]
Type: <class 'list'>
```

x.all(), x.any() # boolean checks 檢查所有或任何元素係咪 *True*。

例子:

```
import torch
x = torch.tensor([True, True, False])
print(f"x: {x}")
print(f"all: {x.all()}")
print(f"any: {x.any()}")
```

運行結果:

```
x: tensor([ True,  True, False])
all: False
any: True
```

x.nonzero() # non-zero indices 返回非零元素嘅索引。

例子:

```
import torch
x = torch.tensor([0, 1, 0, 2, 0, 3])
indices = x.nonzero()
print(f"x: {x}")
print(f"Non-zero indices: {indices.squeeze()}")
```

運行結果:

```
x: tensor([0, 1, 0, 2, 0, 3])
Non-zero indices: tensor([1, 3, 5])
```

x.fill_(val), x.zero_() # in-place 原地填充值或零。下劃線後綴 = 原地。

例子:

```
import torch
x = torch.empty(2, 3)
x.fill_(5.0)
print("After fill_(5.0):")
print(x)
x.zero_()
print("After zero_():")
print(x)
```

運行結果:

```
After fill_(5.0):
tensor([[5., 5., 5.],
        [5., 5., 5.]])
After zero_():
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

x.normal_(), x.uniform_() # random 原地隨機初始化。對權重初始化有用。

例子:

```python
import torch
torch.manual_seed(42)
x = torch.empty(2, 3)
x.normal_(mean=0, std=1)
print("After normal_(0, 1):")
print(x)
x.uniform_(0, 1)
print("After uniform_(0, 1):")
print(x)
```

運行結果:

```
After normal_(0, 1):
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863]])
After uniform_(0, 1):
tensor([[0.8694, 0.5677, 0.7411],
        [0.4294, 0.8854, 0.5739]])
```

`x.add_(y), x.mul_(y)  # in-place ops` 原地算術。直接修改張量節省內存。

例子:

```python
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"Original: {x}")
x.add_(10)
print(f"After add_(10): {x}")
x.mul_(2)
print(f"After mul_(2): {x}")
```

運行結果:

```
Original: tensor([1., 2., 3.])
After add_(10): tensor([11., 12., 13.])
After mul_(2): tensor([22., 24., 26.])
```